



دانشگاه اصفهان
دانشکده مهندسی کامپیوتر

درس انتقال داده

پیوست کار با شبیه‌ساز و API

Operation Fogline Simulator and Student API Guide

استاد درس: دکتر بهروز شاهقلی

دستیار آموزشی: سپهر رجبی

بهار ۱۴۰۵

فهرست مطالب

۳	۱ هدف پیوست و مسیر استفاده
۳	۲ مرز بین شبیه ساز و کد دانشجو
۴	۳ ساختارهای داده ای و مقدارهای معتبر
۵	۴ استراتژی، فایل پیکربندی و ثبت در داشبورد
۷	۵ قرارداد تابع های ماژولی
۸	۶ ترتیب پیشنهادی پیاده سازی
۸	۷ جزئیات پیاده سازی ماژول ها
۸	۱.۷ ساخت قاب
۹	۲.۷ افزودن کنترل خطا
۹	۳.۷ بررسی کنترل خطا
۹	۴.۷ زمان بندی TDM/FDM
۱۰	۵.۷ تصمیم گیرنده
۱۰	۶.۷ بازفرست
۱۰	۷.۷ تطبیق
۱۱	۸ حداقل معیار پذیرش برای هر ماژول
۱۱	۹ داشبورد، راهنمای مرحله ای و توقف
۱۲	۱۰ لاگ، تحلیل و سازگاری
۱۲	۱۱ حداقل دقیق هر ماژول برای دانشجو
۱۳	۱.۱۱ builder Frame
۱۳	۲.۱۱ attach/verify Error-control

۱۳	plan Multiplexing	۳.۱۱
۱۴	decision Receiver	۴.۱۱
۱۴	policy Retransmission	۵.۱۱
۱۴	Adaptation	۶.۱۱

۱۴ نمونه strategy custom معتبر

۱۵ اتصال dashboard به config و ماژولها

۱۵ نمونه log و محاسبه API-level

۱۶ تستهای رفتاری که دانشجو باید قبل از تحویل انجام دهد

۱۶ معیار پذیرش API برای نمره

۱ هدف پیوست و مسیر استفاده

این پیوست برای پیاده‌سازی است. سند اصلی توضیح می‌دهد چرا پروژه انجام می‌شود و دانشجو چگونه باید تحلیل کند؛ این پیوست توضیح می‌دهد کد دانشجو چگونه باید با شبیه‌ساز سازگار باشد. کد دانشجو باید حداقل ماژول‌های لازم را در `student_modules.py` کامل کند و سپس هایی strategy بسازد که در داشبورد قابل انتخاب و قابل آزمون باشند. مسیر اصلی استفاده، داشبورد است:

```
python dashboard.py
```

فرآیند پیشنهادی:

۱. ماژول‌های پایه را در `student_modules.py` کامل کنید.
۲. مطمئن شوید تغییرات شما در `dashboard` و `log` اثر قابل مشاهده دارد.
۳. برای سه وضعیت اصلی، دست‌کم سه مشخصات متفاوت بسازید: نویز طبیعی، تداخل ساخت‌یافته و ظرفیت اضطراری.
۴. یک strategy ساده را به عنوان baseline اجرا کنید و freeze یا ضعف آن را ثبت کنید.
۵. با راهنمای مرحله‌ای، تنظیم ذخیره‌شده بسازید یا پیکربندی موجود را ویرایش کنید.
۶. در زمان توقف، استراتژی را تغییر دهید یا همان استراتژی را با پیکربندی تازه ادامه دهید.
۷. لاگ‌های JSON/CSV را برای گزارش و تحلیل نگه دارید.

۲ مرز بین شبیه‌ساز و کد دانشجو

دانشجو نباید بدنه شبیه‌ساز را برای بهتر شدن امتیاز دستکاری کند. بدنه شبیه‌ساز پیام تولید می‌کند، سناریو را جلو می‌برد، کانال را اجرا می‌کند، رخدادها را اعمال می‌کند، داشبورد و لاگ می‌سازد و ماژول‌های دانشجویی را از طریق قراردادهای مشخص فراخوانی می‌کند. کد دانشجو باید از همین ورودی‌ها استفاده کند و خروجی معتبر برگرداند.

مسیر یا فایل کارکرد

simulator_core.py	ساختارهای داده، نوع‌های شمارشی، وضعیت سیستم، مدل کانال، رخدادها، ظرفیت، لاگ‌گیری و امتیازدهی.
student_modules.py	محل اصلی پیاده‌سازی ماژول‌ها، ثبت پیکربندی‌ها و ذخیره تنظیم‌های دانشجویی.
run_dashboard.py و simulation.py	اجرای داشبورد، ساخت تنظیم، اجرای سناریو و نمایش وضعیت چرخه.
configs/scenarios/*.json	سناریوها، تعداد چرخه‌ها، ظرفیت، پیام‌ها، رخدادهای زنده و آستانه‌های توقف.
configs/custom_strategies.json	محل ذخیره تنظیم‌های ساخته‌شده با راهنمای مرحله‌ای.
story_catalog.py	متن پیام‌های مأموریتی؛ این بخش فقط لایه خوانا است و payload_bits را تغییر نمی‌دهد.
logs/ و reports/	خروجی‌های اجرا، لاگ‌های JSON/CSV و داده‌های تحلیل.

۳ ساختارهای داده‌ای و مقدارهای معتبر

ماژول‌ها باید با ساختارهایی کار کنند که شبیه‌ساز می‌شناسد. خروجی دلخواه یا رشته‌های ناهماهنگ ممکن است اجرا را خراب کند یا باعث شود شبیه‌ساز خروجی ماژول را نادیده بگیرد.

ساختار کاربرد

MissionMessage پیام خام شامل شناسه، مبدأ، مقصد، دسته پیام، اولویت، مهلت، payload_، bits، اندازه بار داده و گاهی payload_text.

Frame قاب ساخته شده از پیام؛ باید بیت های بار داده و فراداده اصلی را حفظ کند.

ProtectedFrame قاب همراه با روش کنترل خطا، بیت های کنترلی و اندازه نهایی.

TransmissionUnit واحد ارسال شامل قاب حفاظت شده و تخصیص TDM/FDM.

ReceivedFrame نتیجه عبور از کانال؛ ممکن است سالم، خراب، قابل تصحیح یا مبهم باشد.

ReceiverDecision تصمیم گیرنده: پذیرش، رد، تصحیح، تصمیم مبهم یا درخواست بازفرست.

RetransmissionDecision تصمیم درباره ارسال دوباره و تعداد تلاش مجاز.

StrategyConfig پیکربندی کامل استراتژی.

مقادیر رایج شامل این مواردند: command، watchtower، radar برای واحدها؛ low، medium، high، critical برای اولویت ها؛ tdm و fdm برای چندبخشی سازی؛ و parity، two_، hamming_secDED و crc32، crc16، checksum16، dimensional_parity برای کنترل خطا.

۴ استراتژی، فایل پیکربندی و ثبت در داشبورد

پیکربندی باید در قالب StrategyConfig ساخته شود و از طریق registry یا فایل configs/ custom_strategies.json قابل بارگذاری باشد. اگر پیکربندی در داشبورد دیده نمی شود، شبیه ساز نمی تواند آن را در اجرا یا توقف انتخاب کند.

```

1 {
2   "name": "student_blackout_priority",
3   "description": "Priority-aware strategy for reduced capacity.",
4   "multiplexing_mode": "tdm",
5   "receiver_strictness": "strict",
6   "use_compact_emergency_mode": true,
7   "suppress_routine_watchtower": true,
8   "watchtower_reports_per_cycle_limit": 1,
9   "error_control_by_priority": {
10     "low": "checksum16",
11     "medium": "crc16",
12     "high": "crc16",
13     "critical": "crc32"
14   },
15   "error_control_by_category": {
16     "emergency_code": "hamming_secDED",

```

```

۱۷     "detection_alert": "crc16",
۱۸     "control_message": "crc16"
۱۹ },
۲۰ "retransmission_limits_by_priority": {
۲۱     "low": 0,
۲۲     "medium": 1,
۲۳     "high": 1,
۲۴     "critical": 2
۲۵ },
۲۶ "tdm_allocation": {"radar": 45.0, "command": 40.0, "watchtower": {15.0,
۲۷ "fdm_allocation": {"radar": 40.0, "command": 40.0, "watchtower": {20.0,
۲۸ "avoid_slots": [],
۲۹ "avoid_bands": [],
۳۰ "auto_adapt": false,
۳۱ "force_equal_fdm": false,
۳۲ "notes": "Use after capacity pressure or emergency backlog appears."
۳۳ }
```

تابع‌های پشتیبان باید این رفتار را فراهم کنند: `list_strategies()` برای نمایش گروهی پیکربندی‌ها، `all_strategy_names()` برای فهرست نام‌های معتبر، `get_strategy(name)` برای برگرداندن یک کپی مستقل از استراتژی، `save_custom_strategy(strategy)` برای ذخیره، و `strategy_to_dict` و `strategy_from_dict` برای تبدیل بین شیء و فایل پیکربندی.

پیکربندی فیلد نقش در پیاده‌سازی

name و description	نام قابل انتخاب در داشبورد و توضیح کوتاه برای دانشجو.
multiplexing_mode	تعیین tdm یا fdm.
error_control_by_priority	روش خطا برای critical, high, medium, low.
error_control_by_category	override برای دسته‌هایی مثل emergency_code یا detection_alert.
retransmission_limits_by_priority	سقف بازفرست بر اساس اولویت.
receiver_strictness	رفتار گیرنده در برابر قاب مشکوک.
tdm_allocation و fdm_allocation	وزن نسبی واحدها؛ این وزن‌ها باید به ظرفیت واقعی نرمال شوند.
avoid_slots و avoid_bands	منابعی که scheduler باید تا حد امکان استفاده نکند.
use_compact_emergency_mode	کاهش سربار برای پیام اضطراری کوتاه، بدون تغییر غیرمجاز. payload.
auto_adapt	فعال کردن adapt_strategy. این بخش اختیاری/extra است، اما در صورت فعال بودن باید StrategyConfig معتبر و قابل توضیح بسازد.

نکته مهم این است که مقدارهای داخل فایل پیکربندی معمولاً رشته هستند، اما در زمان اجرا باید به enum یا مقدار معتبر داخلی تبدیل شوند. همچنین get_strategy(name) باید یک کپی مستقل برگرداند تا تغییرهای زمان توقف، قالب اصلی استراتژی را خراب نکند.

۵ قرارداد تابع‌های ماژولی

شبیه‌ساز تابع‌های مشخصی را فراخوانی می‌کند. نام، ورودی و شکل خروجی باید حفظ شود:

```

۱ def prepare_frame(message, system_state, strategy_config):
۲     ...
۳
۴ def attach_error_control(frame, system_state, strategy_config):
۵     ...
۶
۷ def verify_error_control(received_frame, strategy_config):
۸     ...
۹
۱۰ def choose_multiplexing_plan(protected_frames, system_state, queue_state, strategy_config):

```



```

۱۱     ...
۱۲
۱۳ def decide_received_frame(received_frame, verification_result, system_state, strategy_config):
۱۴     ...
۱۵
۱۶ def decide_retransmission(frame_status, message_context, system_state, strategy_config):
۱۷     ...
۱۸
۱۹ def adapt_strategy(dashboard_snapshot, current_strategy_config, capabilities):
۲۰     ...

```

۶ ترتیب پیشنهادی پیاده‌سازی

برای کاهش خطا، ماژول‌ها را به ترتیب وابستگی پیاده‌سازی کنید:

۱. **قاب‌بندی:** تا وقتی قاب درست ساخته نشود، کنترل خطا و زمان‌بندی قابل اعتماد نیستند.
 ۲. **کنترل خطا:** ابتدا parity/checksum/CRC را درست کنید و سربرار را دقیق گزارش دهید.
 ۳. **بررسی کنترل خطا:** خروجی verification باید برای تصمیم‌گیرنده قابل فهم باشد.
 ۴. **زمان‌بندی:** ظرفیت چرخه، ظرفیت باند، منابع ممنوع و تعویق را رعایت کنید.
 ۵. **تصمیم‌گیرنده:** رفتار گیرنده را با درجه سخت‌گیری و نتیجه verification هماهنگ کنید.
 ۶. **بازفرست:** حد بازفرست، اولویت و مهلت تحویل را وارد تصمیم کنید.
 ۷. **تطبیق:** پس از پایدار شدن ماژول‌های قبلی، تصمیم قاعده‌محور یا تحلیل لاگ را اضافه کنید.
- در هر مرحله یک اجرای کوتاه بگیرید و ببینید تغییر همان ماژول در داشبورد و لاگ‌ها قابل مشاهده است یا نه.

۷ جزئیات پیاده‌سازی ماژول‌ها

۱.۷ ساخت قاب

تابع `prepare_frame` از `MissionMessage` یک `Frame` می‌سازد. باید شناسه پیام، مبدأ، مقصد، دسته، اولویت، مهلت تحویل، شماره تلاش ارسال و `payload_bits` حفظ شوند. متن مأموریتی

نباید به بیت تبدیل شود و نباید جایگزین بار داده فنی شود.

۲.۷ افزودن کنترل خطا

تابع `attach_error_control` روش کنترل خطا را از `StrategyConfig` انتخاب می‌کند. اگر برای دسته پیام قانون جداگانه وجود دارد، آن قانون می‌تواند بر قانون اولویت غلبه کند. خروجی باید روش انتخاب‌شده، بیت‌های حفاظت، سربار، اندازه نهایی و قابلیت تصحیح را مشخص کند. استفاده از `hamming_secded` باید به پیام‌های کوتاه و اضطراری محدود بماند و نباید باعث بریدن بار داده قاب‌های عادی شود.

۳.۷ بررسی کنترل خطا

تابع `verify_error_control` قاب دریافتی را بررسی می‌کند و نتیجه‌ای قابل استفاده برای تصمیم گیرنده می‌سازد. نتیجه می‌تواند سالم بودن، تشخیص خرابی، تصحیح‌شدن، مبهم بودن یا شکست اعتبارسنجی را نشان دهد. این تابع نباید بدون دلیل همه قاب‌ها را معتبر اعلام کند.

۴.۷ زمان‌بندی TDM/FDM

تابع `choose_multiplexing_plan` فهرست قاب‌های حفاظت‌شده را می‌گیرد و تصمیم می‌گیرد کدام قاب در چرخه فعلی ارسال شود. خروجی پیشنهادی باید شامل `mode`، فهرست `units`، دلیل تصمیم، سهم واحدها و مصرف هر واحد باشد. هر واحد باید قاب حفاظت‌شده، بازه زمانی یا باند فرکانسی، و وضعیت تعویق را مشخص کند.

```

1 {
2   "mode": "tdm",
3   "units": [
4     {
5       "protected_frame": pf,
6       "assigned_slot": "slot_3",
7       "assigned_band": None,
8       "deferred": False
9     }
10  ],
11  "reason": "priority allocation with capacity guard",
12  "department_quotas_bits": {"radar": 405, "command": 315, "watchtower": 180},
13  "department_used_bits": {"radar": 384, "command": 300, "watchtower": 160}
14 }
```

در TDM باید بازه‌های زمانی معتبر انتخاب شوند. در FDM باید باند فرکانسی معتبر و ظرفیت هر باند رعایت شود. اگر قاب جا نمی‌شود، باید معوق شود؛ نباید ظرفیت چرخه یا باند به شکل غیرواقعی شکسته شود.

۵.۷ تصمیم گیرنده

تابع `decide_received_frame` نتیجه بررسی کنترل خطا و وضعیت سیستم را می‌گیرد و تصمیم می‌سازد: پذیرش، رد، تصحیح، تصمیم مبهم یا درخواست بازفرست. در سناریوی دوم، پذیرش غلط برای پیام‌های رادار و فرماندهی بسیار پرهزینه است؛ در سناریوی سوم، از دست رفتن مهلت تحویل نیز اهمیت ویژه دارد.

۶.۷ بازفرست

تابع `decide_retransmission` باید اولویت، تعداد تلاش‌های قبلی، مهلت تحویل و حد بازفرست را بررسی کند. بازفرست نامحدود مجاز نیست؛ بازفرست برای پیام بحرانی می‌تواند ارزشمند باشد، اما برای گزارش عادی در لینک شلوغ ممکن است صف را بدتر کند.

۷.۷ تطبیق

تابع `adapt_strategy` زمانی معنا پیدا می‌کند که `config` یا `dashboard` اجازه تطبیق بدهد. ورودی آن `snapshot` از وضعیت سیستم است: سناریو، `freeze cycle`، `reason` ظرفیت، `backlog`، خطاهای کشف‌شده، پذیرش غلط، `miss deadline` و `strategy` فعلی. خروجی باید یک `StrategyConfig` معتبر باشد؛ نه فقط نام یک `config` و نه `dictionary` ناقص.

تطبیق حداقلی می‌تواند `rule-based` باشد. نمونه‌های قابل دفاع: اگر `accepted_incorrect_count > 0` یا `freeze_reason` به پذیرش غلط اشاره دارد، روش `high/critical` را به `CRC16` تغییر دهید و `receiver` را `very strict` کنید. اگر `U_requested > 1` و `backlog` دیدبانی زیاد است، سهم `Watchtower` را کم کنید یا گزارش `routine` را `suppress` کنید. اگر ظرفیت به شدت کاهش یافته است، `emergency compact` و `no-retry` برای `low/medium` را بررسی کنید. تطبیق مبتنی بر یادگیری ماشین هم مجاز است، اما `extra` محسوب می‌شود و باید قابل مقایسه با `baseline` ریاضی باشد.

۸ حداقل معیار پذیرش برای هر ماژول

جدول زیر به دانشجو کمک می‌کند بفهمد یک ماژول فقط اجرا شده یا واقعاً با شبیه‌ساز سازگار است. نبودن این حداقل‌ها معمولاً باعث می‌شود strategy در داشبورد دیده شود اما در رفتار سیستم اثر واقعی نگذارد. نام توابع اجباری در بخش قرارداد تابع‌ها آمده است؛ جدول زیر معیار رفتاری آن‌هاست.

گروه ماژول	حداقل خروجی معتبر	تست رفتاری ساده
قاب‌بندی	قاب باید فراداده و بار داده پیام را حفظ کند و حالت اضطراری را درست جدا کند.	شناسه و اندازه payload در لاگ با پیام اصلی سازگار باشد.
کنترل خطا	قاب حفاظت‌شده باید روش، سربار، بیت‌های حفاظت و اندازه نهایی معتبر داشته باشد.	تغییر روش حفاظت، اندازه قاب و نتیجه بررسی خطا را تغییر دهد.
بررسی خطا	نتیجه باید سالم، تصحیح‌شده، مبهم یا نامعتبر بودن را به شکل قابل استفاده نشان دهد.	قاب خراب همیشه پذیرفته نشود و reason قابل خواندن باشد.
چندبخشی‌سازی	برنامه ارسال باید، mode واحدهای ارسال، quota defer، و مصرف هر واحد را مشخص کند.	تغییر allocation صف و استفاده منابع را تغییر دهد.
تصمیم‌گیرنده	تصمیم باید با نتیجه بررسی خطا، اولویت و سخت‌گیری سازگار باشد.	سخت‌گیری بیشتر پذیرش غلط را کم یا رد شدن را زیاد کند.
بازفرست	تصمیم retry/drop باید سقف تلاش، مهلت و فشار ظرفیت را رعایت کند.	تغییر سقف تلاش تعداد بازفرست و dead-miss line را تغییر دهد.
تطبیق	خروجی باید strategy معتبر باشد؛ در حالت extra، تغییر باید قابل توضیح باشد.	بعد از عبور از threshold، تغییر در چرخه‌های بعدی دیده شود.

۹ داشبورد، راهنمای مرحله‌ای و توقف

راهنمای مرحله‌ای داشبورد می‌تواند نام استراتژی، حالت TDM/FDM، درجه سخت‌گیری گیرنده، کنترل خطا بر اساس اولویت و دسته پیام، حد بازفرست، حالت فشرده اضطراری، کاهش ترافیک دیدبانی، تخصیص ظرفیت و منابع ممنوع را تنظیم کند. در زمان توقف می‌توانید یک استراتژی دیگر انتخاب کنید یا همین پیکربندی را پیش از ادامه تغییر دهید.

مسیر معمول در داشبورد چنین است: فهرست سناریوها را ببینید، استراتژی یا تنظیم ذخیره‌شده

را انتخاب کنید، سناریو را اجرا کنید، در داشبورد چرخه‌ای پیام‌ها و شاخص‌ها را بخوانید، سپس در توقف یا توقف دستی با use، choose، wizard یا edit پیکربندی را تغییر دهید. پس از ادامه اجرا، باید اثر تغییر در چند چرخه بعدی قابل مشاهده باشد.

۱۰ لاگ، تحلیل و سازگاری

شبیه‌ساز لاگ‌های ساختاریافته تولید می‌کند. این لاگ‌ها برای گزارش، دیباگ، مقایسه پیکربندی‌ها و کارهای اختیاری مانند یادگیری ماشین مفیدند. فیلدهای مهم شامل سناریو، چرخه، استراتژی، مبدأ، مقصد، دسته پیام، اولویت، اندازه بار داده، روش کنترل خطا، حالت TDM/FDM، بازه یا باند، وضعیت خرابی، تصمیم گیرنده، تأخیر، سربار، بازفرست، صف عقب‌مانده و دلیل توقف هستند.

برای بررسی سازگاری، این پرسش‌ها را پاسخ دهید:

- آیا استراتژی در داشبورد دیده می‌شود؟
- آیا تغییر کنترل خطا، سربار یا تصمیم گیرنده را عوض می‌کند؟
- آیا تغییر TDM/FDM استفاده از منابع و صف را عوض می‌کند؟
- آیا تغییر تخصیص ظرفیت سهم واحدها را تغییر می‌دهد؟
- آیا تغییر درجه سخت‌گیری گیرنده روی پذیرش و رد اثر دارد؟
- آیا تغییر حد بازفرست تعداد بازفرست و مهلت‌های از دست رفته را تغییر می‌دهد؟
- آیا تغییر استراتژی هنگام توقف در چرخه‌های بعدی دیده می‌شود؟

قاعده سازگاری

اجرای بدون خطا کافی نیست. اگر تغییر پیکربندی در لاگ‌ها، داشبورد یا شاخص‌های عملکرد دیده نشود، احتمالاً خروجی ماژول با شبیه‌ساز هماهنگ نیست یا شبیه‌ساز آن خروجی را استفاده نمی‌کند.

۱۱ حداقل دقیق هر ماژول برای دانشجو

این بخش بر اساس رفتار لازم برای یک پیاده‌سازی سازگار نوشته شده است. هدف، کپی کردن هیچ strategy آماده‌ای نیست؛ هدف این است که چند رفتار پایه واقعاً در اجرا دیده شود و داشبورد بتواند اثر آن‌ها را ثبت کند.

builder Frame ۱.۱۱

prepare_frame باید mode emergency compact را فقط برای MessageCategory . EMERGENCY_CODE فعال کند. در حالت payload compact، باید به ۱۶ بیت محدود/تکمیل شود و method به HAMMING_SECEDED تغییر کند. در حالت معمول، اگر strategy اشتباهاً Hamming را برای frame عادی خواست، باید به CRC۱۶ fallback شود تا truncate payload نشود. se-number quence می‌تواند از id message استخراج شود، ولی باید deterministic باشد و در frame قرار بگیرد.

attach/verify Error-control ۲.۱۱

attach_error_control باید header + payload را محافظت کند، نه فقط payload را. خروجی ProtectedFrame باید protected_bits، overhead_bits، total_size_bits و correction_capable درست داشته باشد. verify_error_control باید همان method ارسال‌شده را verify کند و برای receiver فیلدهای کافی مثل valid، corrected، ambiguous، parity، data_bits و reason فراهم کند. حداقل checksum و CRC۱۶ باید قابل دفاع باشند؛ parity، CRC۳۲ و SECDED دامنه کامل‌تری ایجاد می‌کنند.

plan Multiplexing ۳.۱۱

choose_multiplexing_plan باید بر اساس TDM strategy_config.multiplexing_mode یا FDM را انتخاب کند. allocation خام باید normalize شود؛ اگر همه سهم‌ها صفر یا نامعتبر بود، share equal استفاده شود. سپس quota هر department چنین محاسبه شود:

$$Q_d = \lfloor C_{available} \times share_d \rfloor$$

critical/high اجازه دارند capacity unused را قرض بگیرند، اما low/medium باید با quota محدود شوند. خروجی هر unit باید protected_frame، assigned_slot یا assigned_band و deferred داشته باشد. اگر avoid_slots یا avoid_bands تنظیم شده است، scheduler باید تا حد ممکن از آن‌ها استفاده نکند.

۴.۱۱ decision Receiver

receiver باید بین ACCEPT، CORRECTED، AMBIGUOUS و RETRANSMIT_REQUESTED فرق بگذارد. اگر verification معتبر است اما method برابر SECDED Hamming است و flag channel نشان‌دهنده burst است، در نسخه آموزشی باید ambiguous شود؛ چون SECDED برای تک‌خطا خوب است ولی burst را نباید blind قبول کرد. اگر ambiguous verification است و frame critical/high یا از Radar/Command است، strict strict/very باید آن را unsafe بداند.

۵.۱۱ policy Retransmission

decide_retransmission باید، deadline limit و capacity را همزمان ببیند. اگر attempt به limit رسیده، action باید drop باشد. اگر ظرفیت کاهش یافته و پیام low/medium است، suppression بهتر از storm retry است. اگر cycle از deadline گذشته است، retransmission نباید انجام شود. برای سناریوهای دوم و سوم، limit صفر برای بسیاری از ها priority می‌تواند انتخاب درست باشد، چون score backlog retry را نابود می‌کند.

۶.۱۱ Adaptation

adapt_strategy باید dashboard_snapshot.scenario_id، freeze_reason و metrics را بخواند و StrategyConfig معتبر برگرداند. حالت پایه می‌تواند strategy فعلی را بی‌تغییر برگرداند تا API امن بماند. حالت extra می‌تواند rule-based یا ML باشد. در rule-based، تصمیم باید با هایی threshold مثل $U_{requested}$ ، accepted_incorrect_count، deadline_miss_count و backlog توضیح داده شود. در ML، مدل باید فقط پیشنهاد config بدهد و همچنان خروجی نهایی از قرارداد StrategyConfig پیروی کند.

۱۲ نمونه strategy custom معتبر

دانشجو می‌تواند پیکربندی خود را در configs/custom_strategies.json ذخیره کند. مقادیر enum باید string معتبر باشند.

```

۱ {
۲   "strategies": {
۳     "my_s2_crc16_strict_shaped": {

```

```

۴     "description": "CRC16 for Radar/Command, suppress Watchtower chatter",
۵     "multiplexing_mode": "tdm",
۶     "error_control_by_priority": {
۷         "low": "checksum16", "medium": "checksum16",
۸         "high": "crc16", "critical": "crc16"
۹     },
۱۰    "error_control_by_category": {
۱۱        "detection_alert": "crc16",
۱۲        "control_message": "crc16",
۱۳        "emergency_code": "crc16",
۱۴        "routine_report": "checksum16"
۱۵    },
۱۶    "retransmission_limits_by_priority": {
۱۷        "low": 0, "medium": 0, "high": 0, "critical": 0
۱۸    },
۱۹    "receiver_strictness": "very_strict",
۲۰    "watchtower_reports_per_cycle_limit": 0,
۲۱    "tdm_allocation": {"radar": 50.0, "watchtower": 00.0, "command": {50.0,
۲۲    "fdm_allocation": {"radar": 50.0, "watchtower": 00.0, "command": {50.0,
۲۳    "avoid_slots": ["slot_2"],
۲۴    "avoid_bands": ["band_b"],
۲۵    "notes": "Use after structured-interference freeze."
۲۶    }
۲۷    }
۲۸    }
    
```

۱۳ اتصال dashboard به config و ماژول‌ها

وقتی دانشجو در dashboard روش error، control، allocation strictness یا suppress-sion را تغییر می‌دهد، آن تغییر باید در StrategyConfig قرار بگیرد و سپس توسط ماژول‌ها مصرف شود. برای مثال، اگر dashboard سهم Watchtower را صفر می‌کند اما choose_multiplexing_plan هنوز Watchtower را ارسال می‌کند، او و کد هم‌راستا نیستند. آزمون درست این است که بعد از تغییر config، log نشان دهد department_quotas_bits، department_used_bits، تعداد، ها deferred ها method و sizes frame تغییر کرده‌اند.

۱۴ نمونه log و محاسبه API-level

در این بخش نام ها strategy با برچسب آموزشی نوشته می‌شود، نه با نام دقیق فایل یا تنظیم داخلی. این کار باعث می‌شود جدول‌ها و ها log نقش راهنمای تحلیل داشته باشند و به فهرست

جواب آماده تبدیل نشوند. دانشجو باید همین نوع تحلیل را روی config خودش انجام دهد. در اجرای واقعی یک طرح پایه سناریوی دوم که در این سند با S2-A checksum-baseline نمایش داده می‌شود، چرخه ۱۵ چنین snapshot داشت:

```
strategy_label = S2-A checksum-baseline
strategy_family = checksum_general_under_structured_interference
frames_requested = 18
frames_transmitted = 7
capacity_requested_bits = 3584
capacity_available_bits = 1200
corrupted_critical_frame = true
corruption_type = structured
verification_reason = checksum_ok
accepted_incorrect = true
```

در این خانواده پایه، method همه های frame حساس checksum16 بود. یک frame بحرانی Command با خطای structured توسط receiver با reason برابر checksum_ok پذیرفته شد و accepted_incorrect=True شد. در سطح API، این یعنی verify_error_control و decide_received_frame از نظر قراردادی اجرا شده‌اند، اما policy برای آن سناریو ضعیف بوده است. اصلاح API-level این است که method config را برای high/critical به CRC16 تغییر دهد و strictness receiver را very strict کند.

محاسبه همان چرخه: Command critical با payload هشتاد و checksum16 اندازه $32 + 80 + 128 = 16$ بیت دارد. Radar high با payload شصت و چهار اندازه $32 + 64 + 16 = 112$ بیت دارد. Watchtower routine با payload ۱۹۲ اندازه $32 + 192 + 16 = 240$ بیت دارد. پس ارسال واقعی برابر $2 \times 128 + 4 \times 112 + 1 \times 240 = 944$ بیت بود. اما چون backlog زیاد بود، درخواست کل 3584 بیت شد و $3584/1200 = 2.99$. یک نسخه اصلاح شده از همین خانواده طراحی، ترافیک Watchtower medium را حذف کرد و فقط $2 \times 128 + 4 \times 112 = 704$ بیت حساس فرستاد؛ بنابراین $704/1200 = 0.5867$ و corruption silent صفر شد. این عددها نمونه مسیر تحلیل هستند؛ نام دقیق، سهمیه‌ها و های threshold نهایی باید از اجرای خود دانشجو گزارش شوند.

۱۵ تست‌های رفتاری که دانشجو باید قبل از تحویل انجام دهد

۱. اجرای `pythonvalidate_submission.py` و `python-mpytest-q`.

۲. اجرای هر سه سناریو با seed ثابت و ذخیره JSON/CSV.

۳. اجرای حداقل یک strategy ضعیف و یک strategy اصلاح شده برای نشان دادن قبل/بعد.

۴. بررسی اینکه tdm_allocation و fdm_allocation واقعاً quota را تغییر می‌دهند.
۵. بررسی اینکه avoid_slots و avoid_bands در plan رعایت می‌شوند.
۶. بررسی اینکه receiver_strictness برای frame critical ambiguous اثر دارد.
۷. بررسی اینکه SECDED Hamming برای frame عادی payload را truncate نمی‌کند.
۸. بررسی اینکه adaptation یک StrategyConfig جدید و قابل اجرا برمی‌گرداند، نه فقط string یا dictionary ناقص.

۱۶ معیار پذیرش API برای نمره

برای گرفتن نمره پایه، API باید اجرا شود، سناریوها حداقل ۷۵ بگیرند، و گزارش نشان دهد هر ماژول چگونه به simulator وصل شده است. برای نمره بالاتر، دانشجو باید نشان دهد چرا strat-egy انتخابی از نظر ظرفیت، روش‌های CRC/checksum، انتخاب TDM/FDM، سیاست retrans-strictness mission، گیرنده و تفسیر پیام‌ها بهتر است. برای امتیاز اضافه، پیاده‌سازی‌های حرفه‌ای‌تر فقط وقتی پذیرفته می‌شوند که با همین API پاس شوند و traceability داشته باشند.